

XSectional Momentum Model

Cross-Sectional 12-1 Momentum Strategy | S&P 500 | 2000-2025

Executive Summary | 22 May 2026

1. Executive Summary

XSectional is a quantitative backtesting module that implements the classic Jegadeesh & Titman (1993) cross-sectional momentum strategy on the S&P 500 universe over a 25-year period (January 2000 - December 2025). The system ranks all 500 stocks monthly by their prior 12-month return (skipping the most recent month to avoid short-term reversal), goes long the top 20% of performers and short the bottom 20%, and rebalances at every month-end. It produces a full performance tearsheet including equity curve, annual returns, rolling Sharpe ratio, and drawdown analysis.

2. Strategy Specification

Universe	S&P 500 constituents (current composition via Wikipedia)
Backtest Period	January 2000 - December 2025 (25 years, 300 monthly periods)
Momentum Signal	12-1 momentum: cumulative return over t-13 to t-2 (12 months, skip t-1)
Long Leg	Top 20% of stocks by momentum score - equally weighted (+1/n each)
Short Leg	Bottom 20% of stocks by momentum score - equally weighted (-1/n each)
Rebalancing	Monthly, at month-end close prices
Data Source	Yahoo Finance via yfinance (adjusted close prices, auto-adjusted)
Missing Data	Tickers with >20% missing history dropped; min 10 stocks per leg enforced

3. Technical Architecture

The system is built as a modular Python pipeline. Each module has a single responsibility and communicates through well-defined DataFrame interfaces:

<code>config.py</code>	Central configuration - all tunable parameters in one place (dates, lookahead, quantiles, paths)
<code>data.py</code>	Downloads and caches S&P 500 adjusted close prices from Yahoo Finance; persists to CSV
<code>signals.py</code>	Computes monthly returns and 12-1 momentum scores using log-return rolling sums for numerical stability
<code>portfolio.py</code>	Constructs equal-weighted long-short monthly weights from momentum rankings
<code>backtest.py</code>	Simulates month-by-month portfolio returns; applies weights at t to returns at t+1
<code>report.py</code>	Computes annualised metrics and renders a 4-panel tearsheet PNG
<code>main.py</code>	Orchestrates the full pipeline end-to-end with structured logging

Data flow: `yfinance` -> `data.py` -> `signals.py` -> `portfolio.py` -> `backtest.py` -> `report.py`

4. Performance Metrics Reported

The tearsheet computes and reports the following risk-adjusted metrics (risk-free rate assumed = 0% throughout):

Annualised Return	Geometric mean annualised return over the full backtest period
-------------------	--

Annualised Volatility	Standard deviation of monthly returns scaled to annual frequency ($\times \sqrt{12}$)
Sharpe Ratio	Annualised return divided by annualised volatility
Maximum Drawdown	Largest peak-to-trough decline in the cumulative equity curve
Calmar Ratio	Annualised return divided by the absolute maximum drawdown

The tearsheet PNG contains four panels:

- * Cumulative equity curve (log scale)
- * Year-by-year annual return bar chart (blue = positive, red = negative)
- * Rolling 12-month Sharpe ratio
- * Drawdown chart (shaded area below zero)

5. Academic Foundation

This model replicates the core methodology of Jegadeesh & Titman (1993), the seminal paper establishing that stocks with strong prior returns continue to outperform over intermediate horizons (3-12 months). The one-month skip period is the standard adjustment to avoid the well-documented short-term reversal effect. The long-short portfolio structure isolates the pure momentum factor return, net of market exposure.

- * Jegadeesh, N. & Titman, S. (1993). Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency. *Journal of Finance*, 48(1), 65-91.
- * Reference implementation: Fisjo/momentum-strategy-backtest (GitHub)

6. Technology Stack

Language	Python 3.9+
Data	yfinance, pandas, lxml, requests
Computation	NumPy (log-return rolling sums for momentum)
Visualisation	Matplotlib (Agg backend - headless safe)
Testing	pytest 37 unit tests 100% pass rate
Version Control	Git / GitHub

7. Known Limitations & Future Work

The current prototype is designed for research and proof-of-concept. The following limitations should be disclosed when interpreting backtest results:

- * Survivorship bias - yfinance only returns currently listed tickers; delisted stocks are excluded, which overstates historical returns.
- * Static universe - uses current S&P 500 composition rather than point-in-time historical membership.
- * No transaction costs - bid-ask spread, commissions, and market impact are not modelled.
- * No slippage - assumes execution at exact month-end closing prices.

Planned enhancements:

- * Integrate survivorship-bias-free data (e.g., CRSP or Compustat via WRDS)
- * Add point-in-time index composition using historical S&P 500 membership data
- * Model transaction costs and realistic execution assumptions
- * Extend to multi-factor model (value + momentum + quality)
- * Live monitoring dashboard - agent-based alerting on momentum signal shifts

8. How to Run

Prerequisites: Python 3.9+, dependencies installed via `pip install -r requirements.txt`

```
# Navigate to module directory
cd XSectional/

# Run full pipeline (downloads data on first run, ~2-5 min)
python3 main.py

# Run unit test suite (37 tests, ~5 seconds)
python3 -m pytest -v
```

Output: Performance tearsheet saved to `XSectional/data/tearsheet.png`